

MoneyTransferApp

Application mobile de transfert d'argent

Flutter · Laravel · Ollama · Stripe · Algorand

Louisy David - L3 Informatique

1. Présentation du projet

Qu'est-ce que MoneyTransferApp ?

Application mobile complète de transfert d'argent permettant :

Fonctionnalités	Description
Transfert	Envoyer de l'argent à un autre utilisateur
Recharge	Recharger son compte par carte bancaire
QR Code	Payer en scannant un QR Code "10s"
Historique	Consulter ses transactions avec preuve blockchain
Chatbot	Assistant IA (Ollama LLaMA 3.2)
Profil	Gérer ses informations et son PIN

2. Backend - Laravel 12

Stack technique

Framework Laravel 12 (PHP 8.2)

Base données PostgreSQL 16

Cache Redis 7

Auth Laravel Sanctum

Endpoints 14 routes REST

Soldes Stockés en centimes

Transferts Transaction SQL atomique

Endpoints API (/api/v1/)

POST auth/register — Inscription

POST auth/login — Connexion → Token

POST transfer — Transfert (PIN requis)

POST recharge/create-intent — Stripe

POST recharge/webhook — Stripe Webhook

GET history — Historique paginé

POST qr/generate — Générer QR Code

POST qr/scan — Scanner et payer

POST chatbot/message — Ollama IA

3. Infrastructure - Docker

Container	Image	Rôle	Port
app	PHP 8.2-FPM	Laravel API	:9000
nginx	nginx:1.25-alpine	Serveur web	:8000
postgres	postgres:16-alpine	Base de données	:5432
redis	redis:7-alpine	Cache / Sessions	:6379
ollama	ollama:latest	Chatbot IA LLaMA 3.2	:11434
pgadmin	pgadmin4:latest	Interface DB	:5050

Démarrage en une commande : `./setup.sh`

4. Architecture globale

APPLICATION FLUTTER (Android)

Login · Dashboard · Envoyer · QR Code · Chatbot · Historique

HTTPS + Bearer Token

API LARAVEL 12 (Backend)

Auth · Transfert · Recharge · Historique · QR Code · Chatbot

PostgreSQL · Redis · Ollama LLaMA 3.2

STRIPE

Paiements par carte bancaire
Webhook → Crédit automatique

ALGORAND

Blockchain testnet
Chaque transfert enregistré on-chain

5. Sécurité - 5 couches de protection

Couche
1

Bearer Token Sanctum

Authentification — accès interdit sans token valide

Couche
2

PIN bcrypt (6 chiffres)

Autorisation -> transfert impossible sans PIN, même si le token est volé

Couche
3

Webhook Stripe signé

Anti-fraude -> vérification HMAC-SHA256 de chaque webhook

Couche
4

QR Code TTL 10s + usage unique

Anti-replay -> impossible de réutiliser un QR Code expiré ou déjà scanné

Couche
5

Blockchain Algorand

Immuabilité -> aucune transaction ne peut être falsifiée ou effacée

6. Frontend

Architecture

Pattern : Provider (State Management)

ApiService : appels HTTP + Bearer Token

AuthProvider : état de connexion

TransactionProvider : historique

Au démarrage → vérification du token

Token valide → Dashboard direct

Token invalide → LoginScreen

8 écrans développés

Login / Register

Connexion automatique si token existant

Dashboard

Solde + actions rapides + pull-to-refresh

Envoyer

Email + montant + PIN obligatoire

Recharge

Payment Sheet Stripe native

Historique

Badge Algorand cliquable → Lora

QR Code

Génération TTL 10s + scanner caméra

Chatbot

Ollama LLaMA 3.2 — connaît votre solde

Profil

Modifier infos, MDP et PIN

7. Stripe - Le paiement par carte

- 1 Flutter demande la création d'un PaymentIntent au backend Laravel
- 2 Laravel crée le PaymentIntent via l'API Stripe → reçoit le client_secret
- 3 Flutter affiche la Payment Sheet native Stripe (saisie carte sécurisée)
- 4 Les données de carte vont directement chez Stripe “jamais sur le serveur”
- 5 Stripe confirme le paiement → envoie un Webhook signé à Laravel
- 6 Laravel vérifie la signature HMAC et crédite automatiquement le compte

Carte de test : 4242 4242 4242 4242 · Date : future · CVC : 123

8. Blockchain - Algorand

Pourquoi Algorand ?

Créateur	Silvio Micali (MIT, Prix Turing)
Consensus	Pure Proof of Stake
Vitesse	~4 secondes par bloc
Finalité	Immédiate (pas de rollback)
Frais	~0,001 € par transaction
Usage	CBDC, USDC, finance institutionnelle

Comment ça fonctionne ?

1. Transfert effectué en base SQL
2. Construction TX Algorand
note = {alice→bob, 1€, ref, timestamp}
3. Signature Ed25519 (sodium PHP)
4. Soumission à AlgoNode testnet
5. TX ID stocké + affiché dans Flutter
6. Badge 'Vérifié sur Algorand' cliquable

Transaction vérifiable sur Lora

TX ID	COBUHLC5RRW4EO3FHS3K...
Block	61237003
Timestamp	08 March 2026 18:09:54
Fee	0.001 ALGO
Note	<pre>{ "app": "MoneyTransferApp", "type": "transfer", "amount": "1 EUR", "sender": "alice@test.com", "receiver": "bob@test.com" }</pre>

 lora.algokit.io/testnet/transaction/...

9. Difficultés rencontrées

Stripe — Configuration Android

Le SDK flutter_stripe exige que MainActivity hérite de FlutterFragmentActivity et que le thème utilise Theme.MaterialComponents. Sans ces deux configurations précises, l'application crashait au démarrage avec une PlatformException.

Algorand — Absence de SDK PHP compatible

Le SDK PHP officiel est incompatible avec Laravel 12. J'ai dû implémenter manuellement la signature Ed25519 avec l'extension sodium et l'encodage MessagePack. Les champs binaires (adresses, hashes) doivent être encodés en type Bin et non en String — erreur qui causait une signature invalide.

Docker — Variables d'environnement non rechargées

Les nouvelles clés Stripe dans le .env n'étaient pas prises en compte après un simple 'docker-compose restart'. Il fallait utiliser '--force-recreate' pour que le container recharge les variables d'environnement depuis le fichier .env.